The Journal of Open Source Software

TorchMesh: GPU-Accelerated Mesh Processing for Physical Simulation and Scientific Visualization in Any Dimension

Peter D. Sharpe [1]

1 NVIDIA Corporation, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 11 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

In scientific computing, computer graphics, and engineering simulation, the term "mesh" means different things to different communities: computational fluid dynamics (CFD) and finite element method (FEM) engineers work with volume meshes (3D tetrahedra filling a domain), graphics programmers manipulate surface meshes (2D triangles in 3D space), computer vision researchers process point clouds (0D vertices), and robotics engineers plan paths on curve meshes (1D edges). TorchMesh unifies these diverse concepts by operating on arbitrary-dimensional pure simplicial complexes embedded in arbitrary-dimensional space, all through a single, consistent application programming interface (API). Built on PyTorch (PyTorch Team, 2016), TorchMesh provides end-to-end GPU acceleration, automatic differentiation support, and rigorous mathematical foundations based on discrete exterior calculus (Desbrun et al., 2005; Hirani, 2003). The library enables researchers to seamlessly move mesh operations to GPU, compute gradients through mesh processing pipelines, and work with meshes of any dimensionality using the same code. This dimensional genericity, combined with GPU acceleration and differentiability, distinguishes TorchMesh from existing mesh processing libraries and enables new research directions in physics-informed machine learning, differentiable rendering, and large-scale scientific simulation.

Statement of Need

Mesh processing is fundamental to scientific computing, computer graphics, and engineering simulation. However, existing libraries exhibit a fundamental tradeoff: tools offering rich functionality typically lack GPU acceleration and differentiability (PyVista (Sullivan & Kaszynski, 2019), trimesh (Dawson-Haggerty & contributors, 2019)), while GPU-accelerated differentiable alternatives focus exclusively on graphics applications with 2D surfaces in 3D space (PyTorch3D (Ravi et al., 2020), Kaolin (Jatavallabhula et al., 2019)). No existing tool combines GPU acceleration, dimensional genericity, and automatic differentiation in a unified framework.

PyVista provides an excellent API for mesh manipulation and visualization, but operates exclusively on CPU, lacks support for automatic differentiation, and does not generalize beyond 3D spatial embeddings. For researchers working with large-scale simulations or gradient-based optimization, this CPU-only limitation creates a significant bottleneck. PyTorch3D and Kaolin offer GPU acceleration and differentiability, making them valuable for computer graphics and 3D deep learning, but both are designed specifically for surface meshes in 3D space. They cannot represent 1D curve meshes for path planning, 2D planar meshes for 2D simulations, or 3D tetrahedral volume meshes for FEM/CFD applications. Moreover, neither library implements discrete exterior calculus or provides intrinsic differential operators on manifolds.

TorchMesh fills this gap by providing a dimensionally-generic mesh processing library that

operates on n-dimensional manifolds embedded in m-dimensional spaces (where $n \leq m$), with full GPU acceleration via PyTorch and seamless integration with automatic differentiation. The library implements rigorous discrete exterior calculus (Desbrun et al., 2005; Hirani, 2003) and discrete differential geometry (Meyer et al., 2003), enabling computation of gradients, divergence, curl, and curvature on arbitrary-dimensional manifolds. Crucially, TorchMesh distinguishes between intrinsic (tangent space) and extrinsic (ambient space) differential operators, allowing researchers to compute derivatives along the manifold or in the embedding space as needed.

The restriction to simplicial meshes (i.e., composed exclusively of points, line segments, triangles, tetrahedra, and higher-dimensional simplices) enables rigorous mathematical foundations while maintaining computational efficiency. This design choice allows TorchMesh to implement well-established algorithms from discrete exterior calculus and computational geometry without the ambiguities that arise in mixed-element or polyhedral meshes.

TorchMesh serves multiple research communities: CFD/FEM engineers solving partial differential equations on complex domains, graphics researchers developing differentiable rendering pipelines, machine learning practitioners working with geometric deep learning, and robotics researchers planning paths on manifolds. The library's differentiability enables gradient-based optimization over mesh geometries, making it particularly valuable for inverse problems, shape optimization, and physics-informed neural networks. Its GPU acceleration allows processing of meshes with millions of elements at interactive speeds.

Beyond basic mesh operations, TorchMesh provides sophisticated features including multiple subdivision schemes (linear, Loop (Loop, 1987), and Butterfly (Zorin et al., 1996)), Laplacian smoothing, mesh repair operations, and bounding volume hierarchy (BVH)-accelerated spatial queries. All operations are fully vectorized with no Python loops over mesh elements, ensuring maximum performance on modern GPU hardware.

Key Features

TorchMesh implements a comprehensive suite of mesh processing capabilities:

Discrete Calculus: Gradient and Jacobian computation via least-squares reconstruction and discrete exterior calculus, divergence operators for both methods, curl for 3D manifolds, and intrinsic versus extrinsic derivative modes for manifold-aware computation.

Differential Geometry: Gaussian curvature computation via angle defect method (see Figure 1), mean curvature via cotangent Laplacian (see Figure 2), cell and point normals, and tangent space computation.

Mesh Operations: Linear, Loop (C^2 smooth), and Butterfly (interpolating) subdivision schemes; Laplacian smoothing; uniform remeshing via clustering; and comprehensive repair operations (duplicate removal, orientation fixing, hole filling).

Spatial Queries: BVH construction and queries for point containment and nearest-cell search, with barycentric coordinate interpolation for data sampling.

Data Management: Integration with TensorDict (PyTorch Team, 2023) for structured per-vertex, per-cell, and global data storage with automatic device handling, supporting arbitrary-rank tensor fields including scalars, vectors, and higher-order tensors.

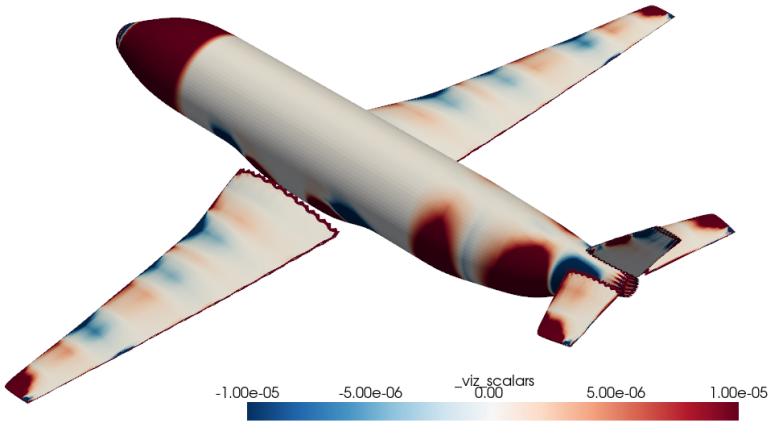

Figure 1: Gaussian curvature on a 3D mesh computed using the angle defect method. Positive curvature (red) indicates convex regions while negative curvature (blue) indicates saddle-like regions.

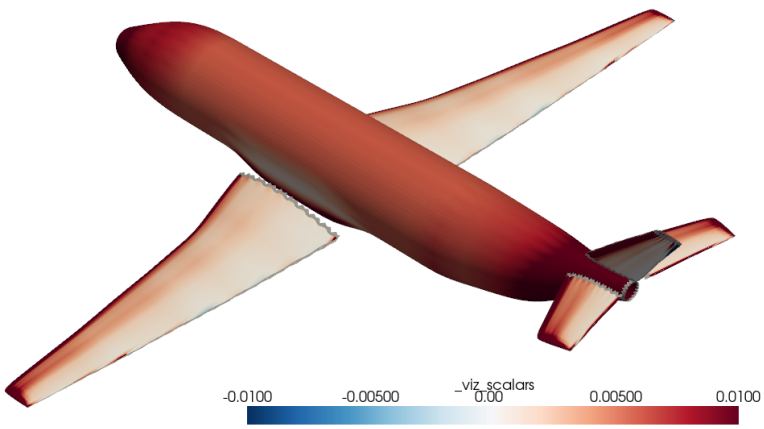

Figure 2: Mean curvature on the same mesh computed via the cotangent Laplacian method. This measure captures the average bending in orthogonal directions.

Acknowledgements

The mathematical foundations of TorchMesh draw heavily from Hirani's seminal work on discrete exterior calculus (Desbrun et al., 2005; Hirani, 2003) and Meyer et al.'s discrete differential geometry operators (Meyer et al., 2003). The API design takes inspiration from PyVista's user-friendly interface (Sullivan & Kaszynski, 2019). The implementation builds upon the PyTorch deep learning framework (PyTorch Team, 2016) and TensorDict library

(PyTorch Team, 2023).

References

Dawson-Haggerty, M., & contributors. (2019). Trimesh: Python library for loading and using triangular meshes. In *GitHub repository*. https://github.com/mikedh/trimesh; GitHub. https://trimesh.org

Desbrun, M., Hirani, A. N., Leok, M., & Marsden, J. E. (2005). Discrete exterior calculus. *arXiv Preprint arXiv:math/0508341*. https://arxiv.org/abs/math/0508341

Hirani, A. N. (2003). *Discrete exterior calculus* [PhD thesis, California Institute of Technology]. https://www.cs.jhu.edu/~misha/Fall09/Hirani03.pdf

Jatavallabhula, K. M., Macklin, M., Litany, O., Fuji Tsang, C., Gojcic, Z., Fidler, S., & others. (2019). *Kaolin: A PyTorch library for accelerating 3D deep learning research*. https://arxiv.org/abs/1911.05063

Loop, C. T. (1987). *Smooth subdivision surfaces based on triangles* [Master's thesis, University of Utah]. https://www.microsoft.com/en-us/research/publication/smooth-subdivision-surfaces-based-on-triangles/

Meyer, M., Desbrun, M., Schröder, P., & Barr, A. H. (2003). Discrete differential-geometry operators for triangulated 2-Manifolds. In H.-C. Hege & K. Polthier (Eds.), *Visualization and mathematics III* (pp. 35–57). Springer. https://doi.org/10.1007/978-3-662-05105-4_2

PyTorch Team. (2016). PyTorch: Tensors and dynamic neural networks in python with strong GPU acceleration. In *GitHub repository*. https://github.com/pytorch/pytorch; GitHub. https://pytorch.org

PyTorch Team. (2023). TensorDict. In *GitHub repository*. https://github.com/pytorch/tensordict; GitHub. https://github.com/pytorch/tensordict

Ravi, N., Reizenstein, J., Novotny, D., Gordon, T., Lo, W.-Y., Johnson, J., & Gkioxari, G. (2020). *Accelerating 3D deep learning with PyTorch3D*. https://doi.org/10.1145/3415263.3419160

Sullivan, C. B., & Kaszynski, A. A. (2019). PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). *Journal of Open Source Software*, *4*(37), 1450. https://doi.org/10.21105/joss.01450

Zorin, D., Schröder, P., & Sweldens, W. (1996). Interpolating subdivision for meshes with arbitrary topology. *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, 189–192. https://doi.org/10.1145/237170.237254

DRAFT